



AI-Based Programming

Lab 2 : Feature Selection with Heuristics & Genetic Algorithms

Eng. Alshaymaa Abdelhady

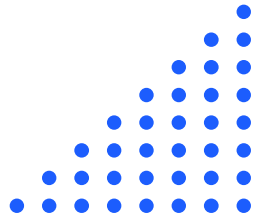
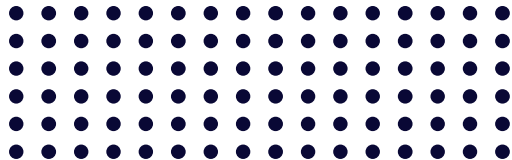




Table of contents



01 Introduction to
Feature Selection

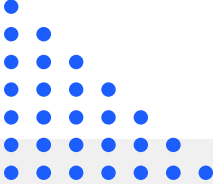
02 Heuristic Feature
Selection

03 Genetic Algorithm
for Feature Selection

04 GA & Heuristic Feature
Selection Implementation

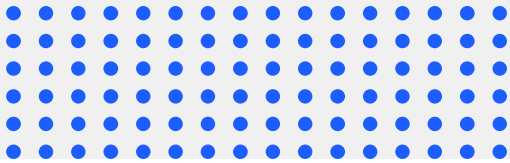
05 Unified Feature Selection
Pipeline

06 Apply Feature Selection
on Dataset





01 Introduction to Feature Selection

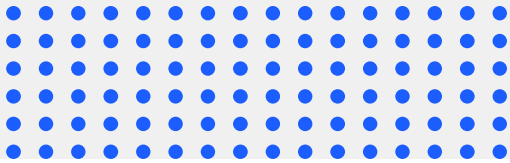


Feature Selection

- Feature selection is the process of choosing only the most useful input features for the model. It helps improve model performance, reduces noise and makes results easier to understand.
- Feature selection methods are essential in data science for several key reasons:
 - **Improved Accuracy:** Models learn better when trained on only important features.
 - **Faster Training:** Fewer features reduce computation time.
 - **Greater Interpretability:** With fewer inputs, understanding model behavior becomes easier.
 - **Avoiding the Curse of Dimensionality:** Reduces complexity when working with high-dimensional data.

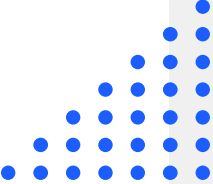


02 **Heuristic Feature Selection**





Heuristic Feature Selection

- A **data preprocessing technique** that selects a “good enough” subset of features from high-dimensional datasets using **rules of thumb, approximations, or simple/nature-inspired algorithms**.
 - Used to **reduce dimensionality, improve training speed, mitigate curse of dimensionality, and enhance model accuracy**.
 - **Key Approaches:**
 - **Wrapper / Hybrid Methods:** Interact with a classifier to evaluate subsets.
 - **Metaheuristics (population-based):** GA, PSO, WOA, GWO, ACO.
 - **Search-based algorithms:** Simulated Annealing, Hill Climbing.
 - **Filter-based heuristics:** Information Gain, Chi-Square.
 - **Advantages:**
 - Faster than exhaustive search.
 - Handles high-dimensional datasets (text, genomics).
 - Improves performance by removing redundant features.
 - Reduces overfitting.
- 
- 

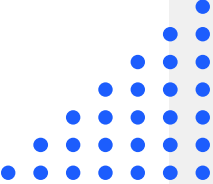


Heuristic Feature Selection (Cont.)

- **Limitations:**

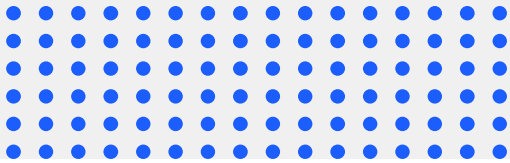
- May not find the global optimum.
- Some methods need careful parameter tuning.
- Wrapper methods can be computationally heavy with complex models.

- **Applications:**

- Text classification (e.g., 20 Newsgroups, large text datasets).
 - Network intrusion detection.
 - Medical data analysis (gene expression, clinical data).
 - Credit risk assessment.
- 
- 



03 Genetic Algorithm for Feature Selection



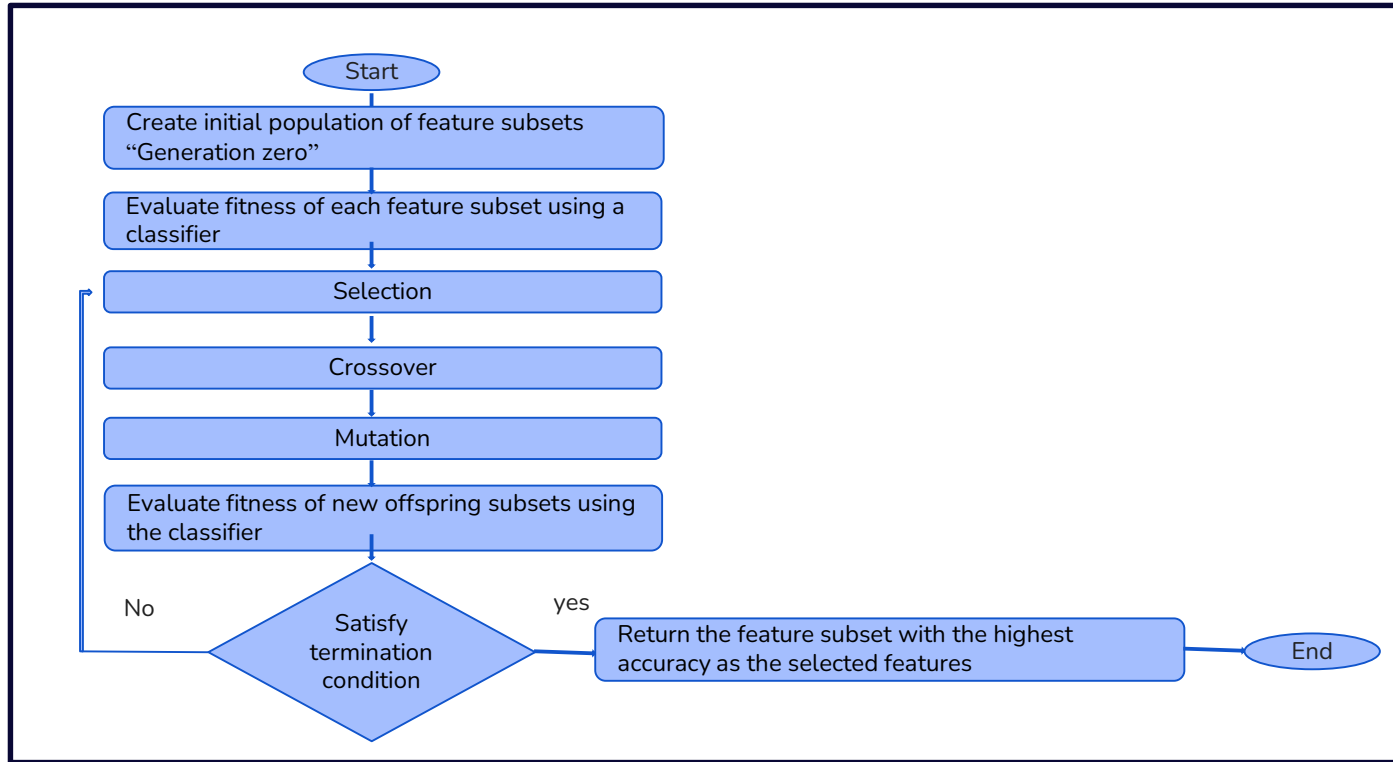
Genetic Algorithm Terminologies

- A **chromosome/ Individual** is a collection of genes.
 - For example, a chromosome can be represented as a binary string where each bit is a gene.
 - 010111010 “Simple binary-coded chromosome”.
- **Population is a collection** of such chromosomes.
 - For example, 010111010, 011011010, 010011011, 010110000 “A population of individuals represented by binary-coded chromosome ”
- **Fitness function**, in every iteration, the individuals are evaluated based on their fitness function.
- Individuals who achieve a better fitness score represent better solutions and are more likely to be chosen to crossover and passed on to the next generation.

Genetic Algorithm for Feature Selection

- Genetic algorithms are **stochastic search algorithms inspired by biological evolution**.
- They mimic **natural selection**: the fittest individuals are chosen to reproduce.
- In **feature selection**, each individual (chromosome) = **subset of features** (binary string).
- Each iteration = **generation**, set of generations = **run**.
- Fitness function = **model performance (accuracy) on the selected features**.
- At the end of a run, we get **highly fit feature subsets**, not optimized model parameters.

Genetic Algorithm Flowchart

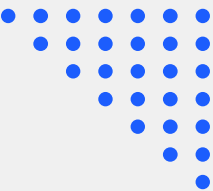


Genetic Algorithm Steps

- **Chromosome Representation:**
 - Represent each chromosome as a **binary vector**, where each bit = 1 if the feature is selected, 0 otherwise.
 - Choose the **population size N**.
- **Define Fitness Function:**
 - Evaluate how well each feature subset performs using a **simple classifier** (e.g., KNN, Decision Tree).
 - Fitness guides which subsets will be chosen for the next generation.
- **Initial Population:**
 - Randomly generate **N feature subsets** as the initial population: $X_1, X_2, \dots, X_N$.
- **Fitness Evaluation:**
 - Calculate the fitness of each feature subset: $f(X_1), f(X_2), \dots, f(X_N)$ based on model performance (accuracy, F1-score, etc.).
- **Selection:**
 - Select **pairs of feature subsets (parents)** from the population.
 - Higher fitness $\rightarrow$ higher probability of being selected.
- **Crossover:**
 - Combine two parent feature subsets to create **offspring**.
 - Mix features from both parents to explore new combinations.

Genetic Algorithm Steps (Cont.)

- **Mutation:**
 - Randomly flip some bits in offspring to introduce **variation**.
 - Helps explore new subsets and avoid local optima.
- **Create Next Generation:**
 - Place offspring feature subsets into the **new population**.
 - Repeat selection, crossover, mutation until the population size = N.
- **Replace Old Population:**
 - Replace the current population with the **newly generated population** of feature subsets.
- **Iteration / Termination:**
 - Repeat **fitness evaluation** → **selection** → **crossover** → **mutation** for multiple generations.
 - Stop when **maximum generations reached** or **no significant improvement** in fitness.
- **Output Selected Features:**
 - Return the **feature subset with the highest fitness** (best performing subset).



Implementation Steps

1. Selection Operator:

- Select the **best-performing feature subsets** from the current population.
- Higher fitness subsets (better accuracy) have a **higher probability** of being chosen as parents.
- Parents are paired to **pass their selected features** to the next generation.

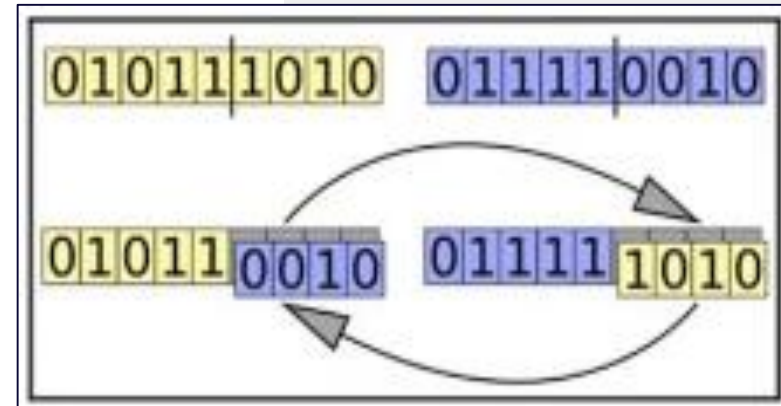


Implementation Steps (Cont.)

- In order to improve the initial population is modified by using:
 - Crossovers
 - Mutation

2. Crossover Operator:

- Combine **two parent feature subsets** to create offspring subsets.
- Offspring inherit features from both parents.
- Helps explore **new feature combinations**



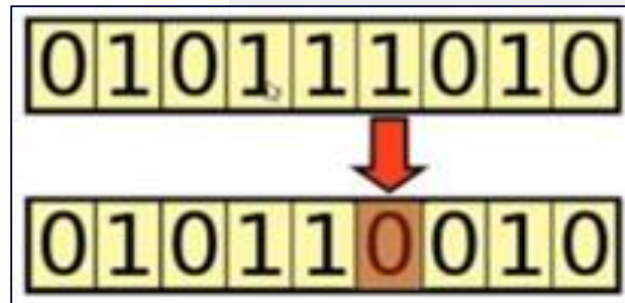
Implementation Steps (Cont.)

3. Mutation:

- Randomly flip bits in a chromosome (feature subset).
- Introduces **new features** into subsets, preventing the algorithm from getting stuck in local optima.

Termination:

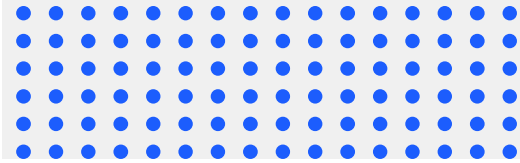
- Stop after a **fixed number of generations** or when **no improvement** in fitness (accuracy) is observed.
- Output: **best-performing feature subset**.





04

GA & Heuristic Feature Selection Implementation



Heuristic Feature Selection Implementation

1. Import Required Libraries

This cell imports all required libraries for feature selection, numerical operations, and evaluation. These libraries will be used to build heuristic and genetic algorithm feature selectors.

```
# Numerical operations
import numpy as np
import random

# Evaluation
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Simple classifier for feature evaluation
from sklearn.neighbors import KNeighborsClassifier
```

Heuristic Feature Selection Implementation (Cont.)

2. Create Feature Evaluation Function

- This function evaluates a given subset of features using a simple classifier (KNN).
- It returns the accuracy score, which will be used as the fitness function.

```
def evaluate_features(X, y, feature_mask):  
    """  
    Evaluate selected features using KNN classifier  
  
    Parameters:  
    X : feature matrix  
    y : labels  
    feature_mask : binary mask indicating selected features  
  
    Returns:  
    accuracy score  
    """  
  
    # Select features  
    selected_features = X[:, feature_mask == 1]  
  
    # Avoid empty feature set  
    if selected_features.shape[1] == 0:  
        return 0  
  
    # Split data  
    X_train, X_test, y_train, y_test = train_test_split(  
        selected_features, y, test_size=0.3, random_state=42  
    )
```

```
# Train classifier  
model = KNeighborsClassifier(n_neighbors=3)  
model.fit(X_train, y_train)  
  
# Predict  
predictions = model.predict(X_test)  
  
# Return accuracy  
return accuracy_score(y_test, predictions)
```

Heuristic Feature Selection Implementation (Cont.)

3. Implement Heuristic Feature Selector (Greedy)

- This heuristic method selects features one by one.
- Each time, it chooses the feature that improves accuracy the most.

```
def heuristic_feature_selection(X, y, max_features=None):  
    """  
    Greedy heuristic feature selection  
    """  
  
    n_features = X.shape[1]  
  
    if max_features is None:  
        max_features = n_features  
  
    selected_mask = np.zeros(n_features, dtype=int)  
  
    best_score = 0  
  
    print("Starting Heuristic Feature Selection...")
```

```
    for _ in range(max_features):  
  
        best_feature = -1  
        best_temp_score = best_score  
  
        for i in range(n_features):  
  
            if selected_mask[i] == 1:  
                continue  
  
            temp_mask = selected_mask.copy()  
            temp_mask[i] = 1  
  
            score = evaluate_features(X, y, temp_mask)  
  
            if score > best_temp_score:  
                best_temp_score = score  
                best_feature = i  
  
        if best_feature == -1:  
            break  
  
        selected_mask[best_feature] = 1  
        best_score = best_temp_score  
  
    print(f"Selected Feature {best_feature}, Accuracy = {best_score:.4f}")  
  
    return selected_mask, best_score
```

GA Feature Selection Implementation

1. Generate Initial Population for GA

- This function creates random feature subsets to form the initial population.

```
def initialize_population(pop_size, n_features):  
  
    population = []  
  
    for _ in range(pop_size):  
        individual = np.random.randint(0, 2, n_features)  
        population.append(individual)  
  
    return np.array(population)
```

GA Feature Selection Implementation (Cont.)

2. Selection Function

- This function selects the best individuals based on fitness scores.

```
def select_parents(population, fitness_scores, num_parents):  
    parents_idx = np.argsort(fitness_scores)[-num_parents:]  
    return population[parents_idx]
```

GA Feature Selection Implementation (Cont.)

3. Crossover Function

- This function combines two parents to create a new child.

```
def crossover(parent1, parent2):  
  
    point = random.randint(1, len(parent1)-1)  
  
    child = np.concatenate([parent1[:point], parent2[point:]])  
  
    return child
```

GA Feature Selection Implementation (Cont.)

4. Mutation Function

- This function randomly flips bits to introduce variation.

```
def mutate(individual, mutation_rate=0.01):  
  
    for i in range(len(individual)):  
  
        if random.random() < mutation_rate:  
            individual[i] = 1 - individual[i]  
  
    return individual
```


GA Feature Selection Implementation (Cont.)

5. Genetic Algorithm Feature Selector

- This function runs the full genetic algorithm process.

```
def genetic_feature_selection(X, y,
                             pop_size=10,
                             generations=10,
                             mutation_rate=0.01):

    n_features = X.shape[1]

    population = initialize_population(pop_size, n_features)

    best_individual = None
    best_score = 0

    print("Starting Genetic Algorithm Feature Selection...")

    for generation in range(generations):

        fitness_scores = []

        for individual in population:
            score = evaluate_features(X, y, individual)
            fitness_scores.append(score)

        fitness_scores = np.array(fitness_scores)

        max_idx = np.argmax(fitness_scores)
```

```
        if fitness_scores[max_idx] > best_score:
            best_score = fitness_scores[max_idx]
            best_individual = population[max_idx]

        print(f"Generation {generation+1}, Best Accuracy = {best_score:.4f}")

        parents = select_parents(population, fitness_scores, pop_size//2)

        new_population = []

        while len(new_population) < pop_size:

            p1, p2 = random.sample(list(parents), 2)

            child = crossover(p1, p2)

            child = mutate(child, mutation_rate)

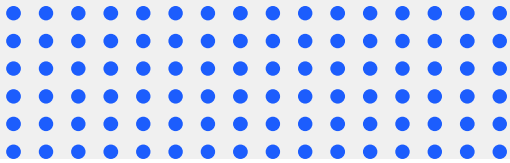
            new_population.append(child)

        population = np.array(new_population)

    return best_individual, best_score
```



05 Unified Feature Selection Pipeline



Feature Selection Pipeline Function

- This function integrates feature selection into a pipeline.

```
def feature_selection_pipeline(X, y, method="heuristic"):

    if method == "heuristic":

        mask, score = heuristic_feature_selection(X, y)

    elif method == "ga":

        mask, score = genetic_feature_selection(X, y)

    else:

        raise ValueError("Method must be 'heuristic' or 'ga'")

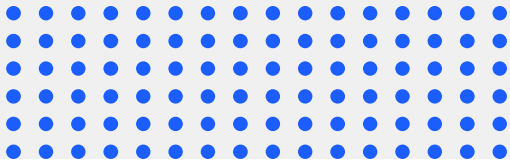
    selected_features = X[:, mask == 1]

    print(f"\nFinal Accuracy: {score:.4f}")
    print(f"Selected Features: {np.sum(mask)}")

    return selected_features, mask, score
```



06 Apply Feature Selection on Dataset



Apply Feature Selection on Dataset

1. GA Feature Selection for Text

- This cell applies Genetic Algorithm to select the most important features from the text dataset. It returns the reduced feature matrix, selected feature mask, and fitness score.

```
X_selected, mask, score = feature_selection_pipeline(X_text, y, method="ga")
```

2. Heuristic Feature Selection for Images

- This cell applies heuristic feature selection to the image dataset. It keeps the most relevant features and removes less important ones to improve efficiency.

```
X_selected, mask, score = feature_selection_pipeline(X_images, y, method="heuristic")
```